PROJECT-BASED LEARNING

Interactive Sorting Algorithm Visualizer

Graphic Era Deemed to be University

Subject: Data Structures and Algorithms

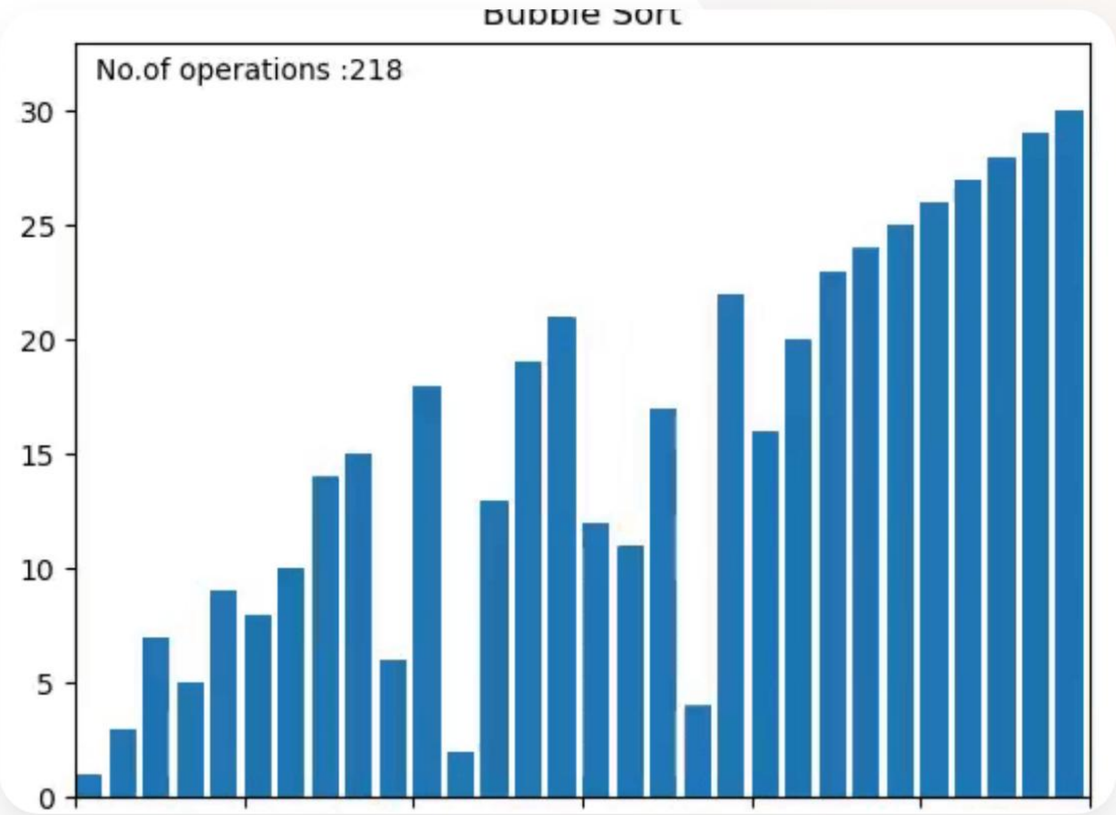
Project Overview & Objective

The Problem

Understanding sorting mechanics through code alone is often abstract and non-intuitive for students. Visualizing swaps and comparisons is crucial for deep learning.

The Solution

A real-time graphical tool built with Python to simulate 6 different sorting algorithms, allowing users to interact with data and observe performance metrics.



Technology Stack

Tkinter

- **Python 3.x:** Core programming language for algorithm implementation.
- **Tkinter:** GUI framework for canvas-based bar representation and user controls.
- **Time Module:** Used for benchmarking execution speed and controlling animation delay.
- **Random Module:** Facilitates random array generation for testing variability.

Core Features & Functionality

Dynamic Input

Users can manually input specific values or choose a "Random Array" option for unpredictable datasets.

Live Animation

Visual bar representation that updates in real-time using colors to highlight current comparisons and swaps.

Performance Timer

Calculates the precise time required to sort the array, allowing for empirical complexity analysis.

Algorithm Suite I: $O(n^2)$

Bubble Sort

Repeatedly swaps adjacent elements if they are in the wrong order. Simple but inefficient for large arrays.

Selection Sort

Finds the minimum element from the unsorted part and puts it at the beginning. Minimizes swaps.

Insertion Sort

Builds the sorted array one item at a time by inserting elements into their correct position.

Note: These algorithms are visualized using a scanning "Red Bar" to indicate active comparison.

Algorithm Suite II: Efficient Sorting

Merge Sort

Uses Divide and Conquer. Splits array into halves, sorts them, and merges back. Highly stable.

Quick Sort

Pivots-based partitioning. Elements are moved around a pivot to sort recursively. Fastest average case.

Heap Sort

Builds a Max-Heap structure and repeatedly extracts the maximum element to sort the array.

Animation & Performance Benchmarking

100+

Array Elements Managed

Real-time Tracking

The visualizer uses **Python's Time module** to capture *Start Time* and *End Time*.

The GUI maintains responsiveness during sorting by invoking **root.update()** within the sorting loop, preventing the window from freezing while the bars move.

Comparative Performance Analysis

Algorithm	Time Complexity	Space Complexity	Performance (Random $N=100$)
Bubble Sort	$O(n^2)$	$O(1)$	~1.45 Seconds
Selection Sort	$O(n^2)$	$O(1)$	~0.85 Seconds
Quick Sort	$O(n \log n)$	$O(\log n)$	~0.04 Seconds
Merge Sort	$O(n \log n)$	$O(n)$	~0.06 Seconds

Implementation Logic

Tkinter Canvas: Each array element is drawn as a rectangle where *height = value*.

Recursion Handling: Complex algorithms like Merge/Quick sort use recursive calls that trigger GUI updates at each swap.

Color Feedback:

- Blue: Unsorted
- Red: Current Comparison
- Green: Sorted Position

			Worst		
earch	Insertion	Deletion	Indexing	Search	Insertion
(n)	-	-	O(1)	O(n)	-
(n)	O(n)	O(n)	O(1)	O(n)	O(n)
(n)	O(1)	O(1)	O(n)	O(n)	O(1)
(n)	O(1)	O(1)	O(n)	O(n)	O(1)
(log(n))	O(log(n))	O(log(n))	O(n)	O(n)	O(n)
(1)	O(1)	O(1)	-	O(n)	O(n)
(log(n))	O(log(n))	O(log(n))	O(n)	O(n)	O(n)
(log(n))	O(log(n))	O(log(n))	-	O(n)	O(n)
(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))
(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))
(log(n))	O(log(n))	O(log(n))	-	O(log(n))	O(log(n))
(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))

Conclusion & Q&A

This project successfully bridge the gap between abstract algorithmic logic and visual perception, demonstrating why $O(n \log n)$ is superior for massive data.

Future Enhancements:

- Adding sound effects for swaps (Auditory Visualization)
- Integrating Searching Algorithms (Binary vs Linear Search)
- Web Deployment using Flask/Django

Thank You!

Image Sources

https://cdn-images-1.medium.com/max/546/1*UgLSkudf_8i9AiprpLNBhw.gif

Source: towardsai.net

---

https://miro.medium.com/v2/resize:fit:1360/1*AIlWq29GeP1eny3wA7aMgA.png

Source: medium.com

---

https://he-s3.s3.amazonaws.com/media/uploads/0728f0e.JPG

Source: www.hackerearth.com